EPTQ: Fast and Accurate 2-bit Post-Training Quantization via Factored E_8 Lattice

Anonymous Author(s)

Abstract

Two-bit Post-Training Quantization (PTQ) offers an attractive path to deploying Large Language Models under strict memory budgets, yet scalar quantization suffers from catastrophic performance collapse at this extreme bit-width. While Vector Quantization (VQ) methods resolve the accuracy issue, existing approaches suffer from practical inefficiencies: complex optimization pipelines result in hours-long quantization, and some methods exhibit severely limited inference throughput. We propose EPTQ, a 2-bit PTQ framework available in two complementary variants (E8 and FE8), achieving high accuracy and practical efficiency through three contributions: (1) **Factored E_8 (FE8) lattice quantization**, which decomposes the E_8 codebook into a compact 4 KB structure that fits in the GPU L1 cache, enabling fast codebook lookup at inference time; (2) **Lightweight scale normalization** via row/column scaling vectors ($\mathbf{g}$, $\mathbf{h}$), which aligns weight distributions to maximize E_8 quantization efficiency, requiring only element-wise per-token scaling at inference; and (3) **Adaptive critical weight preservation** via knee-point threshold detection, which more accurately identifies Hessian-sensitive weights than approaches relying on fixed global criteria, consistently improving perplexity across models, with zero bit-overhead by encoding the preservation mask into the existing codebook index space. Evaluated across multiple LLMs including Llama-2 and Llama-3 families, EPTQ (E8) achieves accuracy competitive with or surpassing QTIP on Llama-2 models, while EPTQ (FE8) surpasses competing VQ methods in decode throughput on most models, achieving up to 38% faster inference than FP16, with both variants reducing quantization time by over 80% compared to competing approaches.

CCS Concepts

• **Computing methodologies** → **Neural networks**; *Neural network models.*

Keywords

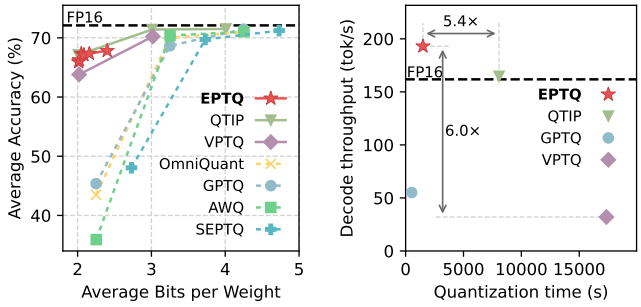
Post-Training Quantization, Vector Quantization, Large Language Models, Model Compression, E_8 Lattice

ACM Reference Format:

Anonymous Author(s). 2026. EPTQ: Fast and Accurate 2-bit Post-Training Quantization via Factored E_8 Lattice. In *Proceedings of Proceedings of the 35th ACM International Conference on Information and Knowledge Management (CIKM '26)*. ACM, New York, NY, USA, 11 pages. https://doi.org/XXXXXXX.XXXXXXX

CIKM '26, Rome, Italy

ACM ISBN 978-1-4503-XXXX-X/26/11
https://doi.org/XXXXXXX.XXXXXXX



(a) Zero-shot Accuracy **(b) Quantization Time vs. Decode Throughput**

Figure 1: Accuracy-efficiency trade-off of 2-bit PTQ methods. (a) Zero-shot accuracy (Llama-2-13B) and (b) quantization time vs. decode throughput (Llama-3-8B). EPTQ achieves competitive accuracy with faster quantization and higher decode throughput than QTIP and VPTQ.

1 Introduction

Can 2-bit post-training quantization (PTQ) achieve both high accuracy and practical efficiency simultaneously? PTQ has become the de facto standard for efficient LLM deployment, yet at the extreme 2-bit regime, existing scalar-based methods such as GPTQ [9], AWQ [13], OmniQuant [20], and SEPTQ [14] suffer from catastrophic *performance collapse*, as illustrated in Figure 1(a), rendering the compressed models practically unusable. Recent Vector Quantization (VQ) approaches resolve the accuracy issue, yet introduce distinct practical limitations: VPTQ [15] incurs both hours-long quantization and severely limited inference throughput, while QTIP [24] similarly requires a complex, hours-long quantization pipeline, as illustrated in Figure 1(b). Such quantization overhead is increasingly costly: the rapid pace of model releases, with each base model spawning numerous fine-tuned variants, demands timely deployment, and practitioners must tune hyperparameters across multiple runs to adapt to each model and target environment.

In this paper, we propose EPTQ (E_8-based Post Training Quantization), a 2-bit PTQ framework available in two complementary variants. EPTQ (E8) uses the full E_8 lattice codebook, achieving accuracy competitive with or surpassing QTIP on Llama-2 models while quantizing approximately 2.5× faster. EPTQ (FE8) introduces Factored-E_8, which compresses the codebook from 1 MB to 4 KB for GPU L1-cache residency, achieving 193.0 tok/s decode throughput on Llama-3-8B, surpassing both QTIP (164.7 tok/s) and FP16 (161.8 tok/s), while completing quantization over 5× faster than QTIP.

The contributions of this paper are as follows:

- **Factored-E_8 (FE8) quantization** (Section 4.1): A compact 4 KB codebook representation of the E_8 lattice that fits in the GPU L1 cache, enabling cache-efficient codebook lookup and inference throughput exceeding FP16.
- **Weight Scale Normalization** (Section 4.2): Iterative row/column scaling via the Sinkhorn-Knopp algorithm that aligns weight distributions to maximize E_8 quantization efficiency, requiring only element-wise per-token scaling at inference.
- **Adaptive Critical Weight Preservation** (Section 4.3): Per-matrix knee-point threshold detection that more accurately identifies Hessian-sensitive weights than fixed-ratio or fixed-threshold approaches; preservation masks are encoded with zero bit-overhead by repurposing a reserved codebook index, eliminating the sparse matrix metadata required by prior methods.
- **Empirical Validation** (Section 5.2): We conduct extensive experiments on Llama-2 and Llama-3 families, demonstrating that EPTQ (E8) matches state-of-the-art 2-bit accuracy, while EPTQ (FE8) achieves the highest decode throughput among all evaluated methods on most models and reduces quantization time by over 80%.

Together, these three components enable a 2-bit PTQ framework that resolves the accuracy-efficiency trade-off. Code and pretrained models will be publicly released upon acceptance.

2 Related Work

We review Post-Training Quantization (PTQ) by categorizing prior work into four paradigms: (1) post-training scalar quantization; (2) post-training vector quantization; (3) distribution adjustment; and (4) critical weight preservation.

2.1 Post-Training Scalar Quantization

Early PTQ research focused on minimizing layer-wise reconstruction error using second-order information. OBQ [7] introduced sequential quantization with inverse Hessian error compensation, which GPTQ [9] extended via a column-wise scheme for efficiency. This framework grounded the sensitivity analysis in subsequent methods like SpQR [5] and SEPTQ [14]. OmniQuant [20] further extends this framework by introducing learnable clipping thresholds and equivalent transformations to reduce quantization error. However, these methods rely fundamentally on Scalar Quantization (SQ), which lacks the geometric flexibility to handle the high-dimensional weight distributions of LLMs, leading to severe performance degradation at 2-bit.

2.2 Post-Training Vector Quantization

Vector Quantization (VQ) overcomes the geometric limitations of scalar quantizers by jointly encoding groups of weights. AQLM [6] represents each weight vector as a sum of multiple learned codebook vectors (additive quantization), achieving high accuracy through iterative joint optimization of codebooks and weight assignments. GPTVQ [25] integrates product quantization into the GPTQ framework, enabling codebook-based quantization without extensive fine-tuning. VPTQ [15] formulates sub-channel VQ as a second-order optimization problem solved channel-independently, with Hessian-weighted k-means for codebook initialization. QuIP# [23] extends QuIP's incoherence processing [3] with E_8 lattice vector quantization, achieving high-accuracy 2-bit LLM compression. QTIP [24] inherits QuIP# while replacing the E_8 lattice with Trellis-Coded Quantization to achieve state-of-the-art 2-bit accuracy. While these approaches advance quantization quality, they face practical efficiency challenges: methods that learn codebooks from data (AQLM, VPTQ) incur substantial quantization time, and QTIP's complex pipeline similarly leads to suboptimal efficiency in both quantization time and inference throughput. EPTQ instead pursues a favorable accuracy-efficiency trade-off through a simpler quantization pipeline that reduces overhead in both quantization and inference, while achieving competitive 2-bit accuracy (Section 5.2).

2.3 Distribution Adjustment for Outlier Mitigation

To facilitate quantization, several studies focus on transforming weight or activation distributions. SmoothQuant [27] and AWQ [13] address activation outliers by migrating quantization difficulty to weights or scaling channels. QuaRot [1] and QuIP [3] apply Hadamard-based rotations to improve distributional incoherence, a technique adopted by QuIP# [23] and QTIP [24]; however, these rotations must be applied to each input token at inference time. EPTQ instead identifies the row- and column-wise variance mismatch as a key obstacle for E_8 lattice quantization and addresses it with lightweight scaling vectors ($\mathbf{g}$, $\mathbf{h}$) derived via the Sinkhorn-Knopp algorithm [21], requiring only element-wise per-token scaling at inference time (Section 4.2).

2.4 Critical Weight Preservation

Recognizing the limitations of uniform quantization, recent methods preserve critical weights based on Hessian sensitivity. SpQR [5] identifies outlier weights using a global sensitivity threshold, keeping those whose quantization error exceeds the threshold in higher precision; the threshold is tuned to preserve approximately 1% of weights across the model. SEPTQ [14] further simplifies this with a static strategy that directly fixes the preservation ratio to the top p% of weights by importance score. Both approaches apply a global criterion that does not adapt to per-layer importance distributions, and both incur additional bit-overhead for storing sparse matrix metadata. EPTQ improves on both fronts: it adopts an adaptive knee-point threshold that automatically determines the preservation ratio per-layer based on the importance score distribution, more accurately identifying the sparse set of truly critical weights; additionally, it encodes the mask with zero bit-overhead by mapping critical weight indices into the existing codebook index space (Section 4.3).

3 Problem Formulation

This paper focuses on the problem of post-training quantization (PTQ) for large language models. The primary goal of PTQ is to compress a full-precision weight matrix while minimizing the impact on the model's output.

Specifically, let $\mathbf{W} \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}$ denote the weight matrix of a linear layer and $\mathbf{X} \in \mathbb{R}^{d_{\text{in}} \times n}$ represent a matrix of n input samples. The objective of PTQ is to find a quantized weight matrix $\widehat{\mathbf{W}}$ that minimizes the squared error of the layer's output. This can be

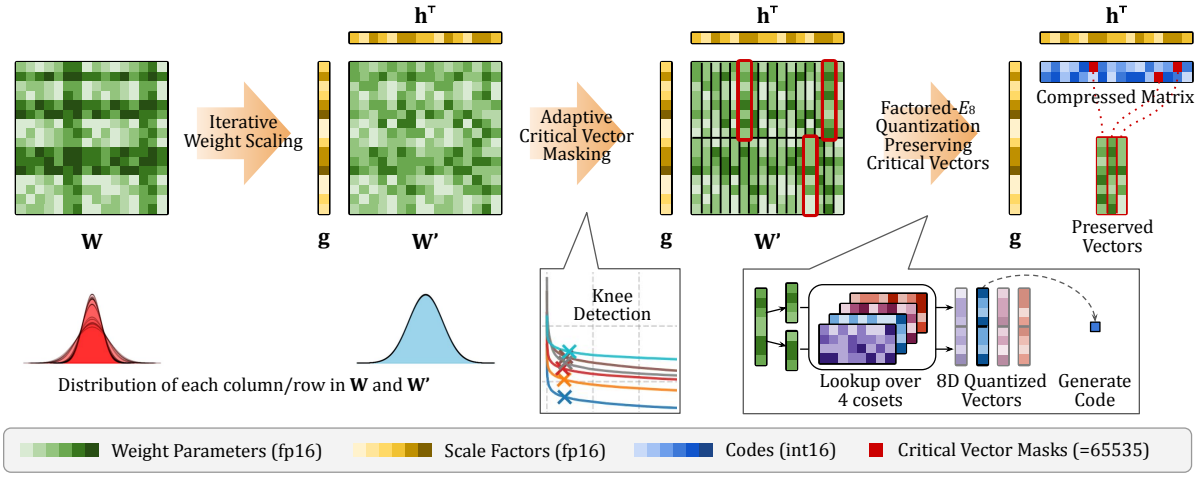


Figure 2: The overall process of EPTQ. Given the original weight matrix W, EPTQ first applies Iterative Weight Scaling to produce the normalized matrix W′ and scale vectors g and h, ensuring uniform weight distribution across all rows and columns to maximize Factored-E_8 efficiency. Critical vectors are then identified via a per-matrix adaptive threshold (Adaptive Critical Vector Masking) and preserved in full precision. The remaining vectors are quantized using Hessian-aware Factored-E_8 Lattice Quantization with sequential error compensation, yielding the compressed matrix.

formally expressed as:

$$\arg\min_{\widehat{\mathbf{W}}} ||\mathbf{WX} - \widehat{\mathbf{W}}\mathbf{X}||_F^2 \quad \text{s.t.} \quad \widehat{\mathbf{W}} = \text{quant}(\mathbf{W}) \tag{1}$$

where $||\cdot||_F$ is the Frobenius norm, and quant$(\cdot)$ is a quantization function, such as the standard Round-To-Nearest (RTN) operator, which maps continuous weight values to a discrete set of quantized levels. That is, our goal is to design a new quant$(\cdot)$ function that transforms $\mathbf{W}$ into $\widehat{\mathbf{W}}$ to minimize Eq. (1), under the constraint that $\widehat{\mathbf{W}}$ is represented using a low number of bits per parameter.

4 Proposed Method

To achieve high-accuracy 2-bit compression, we propose E_8-based Post Training Quantization (EPTQ), which integrates three innovations:

- **Factored-E_8 VQ (Section 4.1):** Exploits a novel factored representation of the E_8 lattice to achieve 2^{16}-point, 2-bit-per-weight capacity with a 4 KB codebook footprint, enabling L1-cache residency and substantially faster inference.
- **Weight Scale Normalization (Section 4.2):** Applies iterative scaling to normalize variance disparities across rows and columns, maximizing the efficiency of E_8 lattice quantization.
- **Adaptive Critical Weight Preservation (Section 4.3):** Determines a per-matrix threshold via knee-point detection to identify Hessian-sensitive weights, and preserves them with zero bit-overhead by mapping the mask to a reserved codebook index.

The integration of these components forms the complete EPTQ algorithm (Section 4.4), as illustrated in Figure 2.

4.1 Factored-E_8 Quantization

Conventional Round-To-Nearest (RTN) quantization performs scalar quantization on individual weights. When considering groups of weights as n-dimensional vectors, RTN places quantization points on a Cartesian grid, a structure that conflicts with the underlying weight distribution. Neural network weights follow a Gaussian distribution, so their n-dimensional representations cluster within a hypersphere centered at the origin. Grid corners are rarely populated, yielding an inefficient use of quantization resources.

Vector Quantization (VQ) with lattice structures addresses this by aligning quantization points with the spherical geometry of the weight distribution. The E_8 lattice, which provides the densest known sphere packing in 8 dimensions [26], is the optimal choice for 8-dimensional VQ and has been applied to LLM weight quantization [23]. However, a naive implementation stores all 2^{16} 8-dimensional lattice points explicitly, requiring 1 MB — too large for the GPU L1 cache, limiting inference throughput due to cache-miss latency in codebook lookups.

We propose **Factored-E_8**, which exploits a factored representation of E_8 to reduce the codebook to 4 KB while keeping all quantization points exactly on the E_8 lattice.

4.1.1 E_8 Lattice and Its Factored Representation. The E_8 lattice is defined as:

$$E_8 = \left\{ x \in \mathbb{Z}^8 \cup (\mathbb{Z} + \tfrac{1}{2})^8 \;\middle|\; \mathbf{1}^\top x \equiv 0 \pmod 2 \right\}$$

We split each 8-dimensional vector $x \in E_8$ into two 4-dimensional halves $[a; b]$ and classify each 4D half by two binary attributes: whether its coordinates are integers or half-integers, and whether the coordinate sum is even or odd. This yields four *coset types*:

$$F_t = \begin{cases} \{a \in \mathbb{Z}^4 \mid \mathbf{1}^\top a \equiv 0 \pmod 2\} & t = 0 \text{ (even full)} \\ \{a \in (\mathbb{Z} + \frac{1}{2})^4 \mid \mathbf{1}^\top a \equiv 0 \pmod 2\} & t = 1 \text{ (even half)} \\ \{a \in \mathbb{Z}^4 \mid \mathbf{1}^\top a \equiv 1 \pmod 2\} & t = 2 \text{ (odd full)} \\ \{a \in (\mathbb{Z} + \frac{1}{2})^4 \mid \mathbf{1}^\top a \equiv 1 \pmod 2\} & t = 3 \text{ (odd half)} \end{cases}$$

A key observation is that the E_8 lattice decomposes *exactly* as the disjoint union of same-type 4D products:

$$E_8 = \bigsqcup_{t=0}^{3} F_t \times F_t \tag{2}$$

This follows directly from the E_8 definition: for any $[a; b] \in E_8$, the 8-dimensional coordinate sum $\mathbf{1}^\top [a; b] = \mathbf{1}^\top a + \mathbf{1}^\top b$ must be even, so a and b must belong to the same parity class (both even-sum or both odd-sum), and they must share the same coordinate type (both integer or both half-integer). Hence $[a; b]$ lies in $F_t \times F_t$ for exactly one t.

We exploit this decomposition to construct a compact codebook. For each type t, we retain the 128 closest points to the origin from F_t, yielding a codebook $C \in \mathbb{R}^{4\times128\times4}$ that stores only $4\times128 = 512$ four-dimensional vectors (4 KB), where $\mathbf{c}_{t,i} := C[t, i] \in \mathbb{R}^4$ denotes the i-th stored vector of type t. The corresponding 8-dimensional quantization points are:

$$\mathbf{c}_{t,i,j} = [\mathbf{c}_{t,i}\ ;\ \mathbf{c}_{t,j}], \quad t \in \{0, 1, 2, 3\},\ i, j \in \{0, \ldots, 127\}$$

By construction (Eq. (2)), every $\mathbf{c}_{t,i,j}$ lies exactly on the E_8 lattice. The product structure yields $4 \times 128 \times 128 = 65{,}536$ distinct 8-dimensional quantization points, matching the full E_8 codebook capacity, at 256× lower storage cost.

4.1.2 Quantization Function and Decomposed Search. Given an 8-dimensional weight vector $\mathbf{v} = [\mathbf{a}; \mathbf{b}]$ (each half 4-dimensional), the quantization function is:

$$\text{fe8quant}(\mathbf{v}, \alpha, C) = \arg\min_{t,i,j} \|\mathbf{v} - \alpha \cdot \mathbf{c}_{t,i,j}\|_2^2 \tag{3}$$

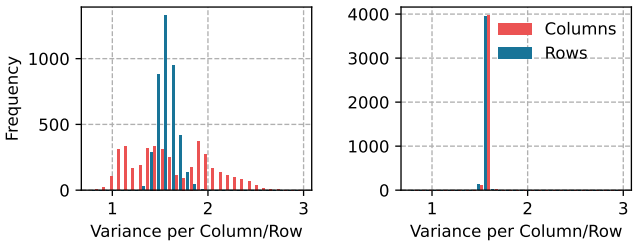
where $\alpha = q_{0.95}(|\mathbf{w}_g|)/r$ is a per-column scale factor, with $q_{0.95}(|\mathbf{w}_g|)$ the 95$^{\text{th}}$ percentile of the column's absolute values (chosen for outlier-robust normalization) and r a codebook radius parameter. Under a Gaussian weight assumption, matching the data and codebook radii yields $r \approx 1.77$; we adopt $r = 1.75$ as a robust default that reliably achieves near-optimal performance across diverse hyperparameter settings (Section 5.3.3).

Naively enumerating all 65,536 candidates is expensive. Because sub-vectors $\mathbf{a}$ and $\mathbf{b}$ share only the type index t, the minimization decomposes as:

$$(t^*, i^*, j^*) = \arg\min_t \left[\min_i \|\mathbf{a} - \alpha\, \mathbf{c}_{t,i}\|^2 + \min_j \|\mathbf{b} - \alpha\, \mathbf{c}_{t,j}\|^2 \right]$$

For each type t, the 128 sub-distances are computed via the proxy-distance identity $\|\mathbf{v} - \mathbf{c}\|^2 = -2(\mathbf{v} \cdot \mathbf{c}) + \|\mathbf{c}\|^2 + \text{const}$, evaluated as a single matrix–vector product. The type minimizing the total distance is selected, requiring only $4\times128\times2 = 1{,}024$ inner products in total.

Each quantized vector is stored in a single 16-bit integer: index = $(t \ll 14) \mid (i \ll 7) \mid j$. Decoding requires only two 4-dimensional table lookups from the 4 KB codebook, which fits entirely in the GPU L1 cache. In contrast, the full E_8 codebook (1 MB) resides in the slower L2 cache; across the many layers of a transformer model, repeated evictions cause substantial memory-access overhead. Consequently, Factored-E_8 more than doubles the decode throughput of a direct E_8 implementation at the full-model level, even surpassing FP16 throughput by up to 38% (Section 5.2).



(a) Original weight matrix W **(b) Scaled weight matrix W′**

Figure 3: The frequency distribution of the variance per row/column for the `out_proj` weight matrix of the 8th layer in the OPT-6.7B model.

4.1.3 Quantizing the Model Weights. The core of our method involves quantizing the model weights $\mathbf{W}$ by dividing them into 8-dimensional vectors $\mathbf{v}$ and applying the fe8quant function (Eq. (3)). Simply applying this function independently across all weight blocks, however, leads to high cumulative quantization error, resulting in severe performance degradation. To mitigate this, we adopt a Hessian-based error compensation mechanism [8]. When a weight vector $\mathbf{v}$ is quantized to $\widehat{\mathbf{v}}$, an important characteristic of the compensation mechanism is its *row independence*: the error from quantizing $\mathbf{W}_{i,j}$ propagates only within row i, leaving other rows unaffected. This independence is reflected in the optimal update $\Delta\mathbf{W}$ applied to the unquantized weights $W_{i,k}$ (where $k \neq j$):

$$\Delta\mathbf{W}_{i,k} = -\frac{\mathbf{W}_{i,j} - \widehat{\mathbf{W}}_{i,j}}{\mathbf{H}^{-1}_{j,j}} \mathbf{H}^{-1}_{j,k} \tag{4}$$

where $\mathbf{H} = 2\mathbf{X}\mathbf{X}^\top$ is the Hessian matrix. This column-wise structure allows us to directly adopt the efficient block-wise processing and Cholesky decomposition of GPTQ [9], ensuring computational scalability for large models. The full quantization procedure incorporating this Hessian-based compensation is reflected in lines 5–17 of Algorithm 3, where lines 5–6 compute the Hessian inverse and lines 8–17 perform the column-wise quantization with error propagation.

4.2 Weight Scale Normalization

The high efficiency of Factored-E_8 quantization (Section 4.1) relies on the theoretical assumption that the input weight parameters follow a Gaussian distribution. While the entire weight matrix $\mathbf{W}$ appears to follow a single Gaussian distribution, we found that the weights have quite different distributions when checked row by row and column by column. Specifically, although the mean of the weights in most rows and columns remains close to zero, the variance is varied greatly (see Figure 3a). If we apply Factored-E_8 quantization directly to this uneven matrix, the quantization error will be much higher in some areas than others, compromising the intended efficiency of the lattice-based quantization.

To address this issue, we define our scale normalization problem as finding the optimal vectors $\mathbf{g}$ and $\mathbf{h}$ that allow the weight matrix $\mathbf{W}$ to be factorized as:

$$\mathbf{W} = \mathbf{W}' \odot (\mathbf{g}\mathbf{h}^\top)$$

where $\mathbf{W}'$ is the normalized matrix, and $\odot$ is element-wise multiplication. Here, $\mathbf{g}$ and $\mathbf{h}$ play as row-wise and column-wise scale vectors, respectively. The objective is to ensure that the variance of the elements in $\mathbf{W}'$ becomes uniform across all rows and columns (see Figure 3b). The quantization procedure is then performed on this normalized matrix $\mathbf{W}'$.

Since the mean of weights is near zero across rows and columns, the variance of each row and column is well approximated by its mean square, and variance equalization reduces to an L_2 normalization problem. However, L_2 norms are sensitive to outliers: a single large weight would disproportionately inflate the row or column scale, forcing the majority of normal weights to be over-compressed. We instead normalize L_1 norms (mean absolute values), which are more robust to outliers and allow weights that deviate significantly from zero to remain so. This is by design: such outlier weight vectors are identified and preserved in FP16 by the Critical Weight Preservation mechanism (Section 4.3), making aggressive outlier suppression during normalization unnecessary. Under the Gaussian weight assumption, the mean absolute value is proportional to the standard deviation ($\mathbb{E}[|w|] = \sigma\sqrt{2/\pi}$), so L_1 equalization directly serves the variance unification objective. We therefore seek $\mathbf{g}$ and $\mathbf{h}$ such that the mean absolute value of every row and column in $\mathbf{W}'$ converges to unity.

This problem, which aims to normalize both the row and column sums (L_1 norms) to a target value, is mathematically equivalent to computing a *Doubly Stochastic Matrix* that is known to be solved efficiently by the Sinkhorn-Knopp algorithm [21]. Thus, we modify the Sinkhorn-Knopp algorithm and apply this iterative scaling process to the absolute magnitude matrix $|\mathbf{W}|$ to determine the optimal vectors $\mathbf{g}$ and $\mathbf{h}$. The procedure alternates between row and column normalization steps until the scale factors stabilize. Consequently, the variances across all rows and columns are effectively equalized, as shown in Figure 3b. Notably, this iterative process is highly efficient, typically converging in fewer than 5 iterations. The specific steps are detailed in Algorithm 1.

Algorithm 1: Iterative Weight Scaling

Input: The weight matrix $\mathbf{W}$
Output: The scaled matrix $\mathbf{W}'$, the row scale vector $\mathbf{g}$, the column scale vector $\mathbf{h}$

$\mathbf{g} \leftarrow \mathbf{1}_{d_{\text{row}}}$
$\mathbf{h} \leftarrow \mathbf{1}_{d_{\text{col}}}$
repeat
  $\mathbf{g} \leftarrow \left(|\mathbf{W}| \cdot \text{diag}(\mathbf{h})^{-1}\right) \cdot \mathbf{1}_{d_{\text{col}}} / d_{\text{col}}$
  $\mathbf{h} \leftarrow \left(|\mathbf{W}|^\top \cdot \text{diag}(\mathbf{g})^{-1}\right) \cdot \mathbf{1}_{d_{\text{row}}} / d_{\text{row}}$
until *convergence*
$\mathbf{W}' \leftarrow \mathbf{W} \oslash (\mathbf{g}\mathbf{h}^\top)$ // $\oslash$: element-wise division

4.3 Adaptive Critical Weight Preservation

Not all weight parameters in a neural network contribute equally to the final model performance; certain critical weights exhibit significantly higher sensitivity than others [12]. Preserving these critical weights by excluding them from the quantization process is effective for maintaining model performance [5, 14]. Applying this principle, since our E_8 lattice quantization groups weights into

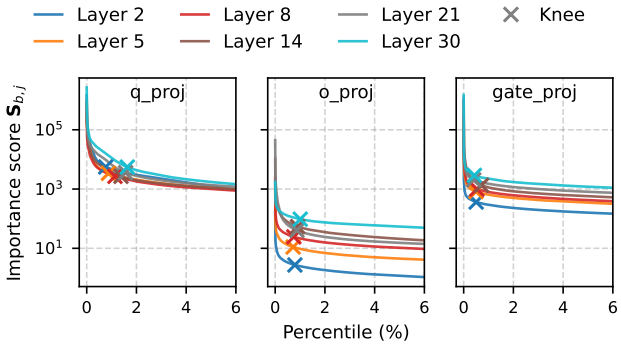


Figure 4: Importance score distributions (top 6%) across layers for three sublayer types in LLaMA-3-8B; × marks the per-layer knee point. In `o_proj`, score scales differ by orders of magnitude across layers, so a fixed threshold captures vastly different fractions. In `q_proj`, scales are similar but knee positions vary in percentile terms, so a fixed ratio misses the natural transition. An adaptive per-matrix threshold handles both.

8-dimensional vectors, we compute an importance score for each 8-dimensional vector and identify the critical vectors via an *adaptive per-matrix threshold* that captures the unique sensitivity profile of each matrix.

To select the most critical 8-dimensional vectors for preservation, we must quantify the importance of each vector. We define the importance score $\mathbf{S}_{b,j}$ for the vector $\mathbf{v}_{b,j} = \mathbf{W}_{8b:8(b+1),j}$ based on the output loss change given an input matrix $\mathbf{X}$ when that vector is quantized:

$$\mathbf{S}_{b,j} = \min_{\widehat{\mathbf{W}}(b,j)} \left\| \mathbf{W}\mathbf{X} - \widehat{\mathbf{W}}(b,j)\mathbf{X} \right\|_F^2$$

where $\widehat{\mathbf{W}}(b,j)$ is the matrix where only the vector $\mathbf{v}_{b,j}$ has been quantized, and the remaining unquantized weights have been updated. Leveraging the row independence of the compensation mechanism (established in Section 4.1) and solving the constrained optimization problem using Lagrange Multipliers (the detailed proof is in Appendix A), the importance score $\mathbf{S}_{b,j}$ is derived as follows:

$$\mathbf{S}_{b,j} = \frac{\left\| \mathbf{v}_{b,j} - \widehat{\mathbf{v}}_{b,j} \right\|_2^2}{[\mathbf{X}\mathbf{X}^\top]^{-1}_{j,j}} \tag{5}$$

where $\widehat{\mathbf{v}}_{b,j}$ is the quantized vector of $\mathbf{v}_{b,j}$. We use this equation to compute the importance score of each 8-dimensional vector in $\mathbf{W}$ using a particular input matrix $\mathbf{X}$.

Given a threshold τ, we construct the binary mask matrix $\mathbf{M}$ using the Kronecker product ($\otimes$):

$$\mathbf{M} = \mathbf{1}_8 \otimes \mathbf{I}(\mathbf{S} \geq \tau) \tag{6}$$

where $\mathbf{1}_8$ is an 8-dimensional column vector of ones, $\mathbf{S}$ is the matrix of all importance scores $\mathbf{S}_{b,j}$, and $\mathbf{I}(\cdot)$ is the indicator function. Each 8-dimensional vector $\mathbf{v}_{b,j}$ whose importance score meets or exceeds τ is preserved in full precision, while the remaining vectors are quantized.

Algorithm 2: Critical Vector Masking

Input: The weight matrix $\mathbf{W}$, the calibration dataset $\mathbf{X}$, the codebook C, search ratio ρ, codebook radius r
Output: The binary mask matrix $\mathbf{M}$

$\mathbf{S} \leftarrow \mathbf{0}_{(d_{\text{row}}/8)\times d_{\text{col}}}$
for $j = 0, \cdots, d_{col} - 1$ **do**
  $\alpha_j \leftarrow q_{0.95}(|\mathbf{W}_{:,j}|)/r$
  for $b = 0, \cdots, d_{row}/8 - 1$ **do**
    $\mathbf{v} \leftarrow \mathbf{W}_{8b:8(b+1),j}$
    $\widehat{\mathbf{v}} \leftarrow \text{fe8quant}(\mathbf{v}, \alpha_j, C)$
    $\mathbf{S}_{b,j} \leftarrow \frac{\|\mathbf{v}-\widehat{\mathbf{v}}\|_2^2}{[\mathbf{X}\mathbf{X}^\top]^{-1}_{j,j}}$ // Eq. (5)
// Knee-point threshold detection [19]
$\{s_1 \geq \cdots \geq s_N\} \leftarrow \text{sort}(\text{flatten}(\mathbf{S}), \text{desc})$
$K \leftarrow \max(2, \lfloor N\rho \rfloor)$
for $k = 1, \cdots, K$ **do**
  $x_k \leftarrow (k-1)/(K-1)$
  $y_k \leftarrow (\log s_k - \log s_K) \,/\, (\log s_1 - \log s_K)$ // Eq. (7)
$k^* \leftarrow \text{argmax}_k\ (1 - x_k) - y_k$
$\tau \leftarrow s_{k^*}$ // Eq. (8)
$\mathbf{M} \leftarrow \mathbf{1}_8 \otimes \mathbf{I}(\mathbf{S} \geq \tau)$ // Eq. (6)

The key question is how to set τ. As shown in Figure 4, the importance score distributions exhibit a *heavy-tail* structure, but layers differ in two distinct ways: the absolute score scale can vary by orders of magnitude across layers (e.g., `o_proj`), and the knee location in percentile terms also shifts across layers (e.g., `q_proj`). Prior methods apply a global criterion that fails to handle either case: SEPTQ [14] preserves a fixed fraction of weights per layer, thus missing the natural knee when its percentile position varies; SpQR [5] applies a fixed score threshold across all layers, breaking down when the absolute score scale differs. We instead determine τ adaptively per matrix using the Kneedle algorithm [19].

Concretely, let $\{s_1 \geq s_2 \geq \cdots \geq s_N\}$ denote the $N = (d_{\text{row}}/8) \times d_{\text{col}}$ importance scores sorted in descending order. We restrict attention to the top-K scores with $K = \max(2, \lfloor N\rho \rfloor)$ (default $\rho = 0.1$), which capture the steep descent region of the distribution. For each rank $k \in \{1, \ldots, K\}$, we define normalized rank $x_k = (k-1)/(K-1)$ and normalized log-score:

$$y_k = \frac{\log s_k - \log s_K}{\log s_1 - \log s_K} \tag{7}$$

The resulting curve (x_k, y_k) is concave and decreasing, lying below the diagonal $y = 1 - x$. The Kneedle distance $\delta_k = (1 - x_k) - y_k$ is maximized at the knee point:

$$k^* = \underset{k}{\text{argmax}}\ \delta_k, \qquad \tau = s_{k^*} \tag{8}$$

This per-matrix τ automatically identifies the transition point from high-sensitivity to low-sensitivity blocks without any per-matrix manual tuning, producing a distinct critical weight ratio for each sublayer. The complete process of computing the importance scores and constructing the mask is detailed in Algorithm 2.

Figure 5 illustrates the resulting critical weight ratios across transformer layers on LLaMA-3-8B, confirming that each sublayer develops a distinct per-layer profile. A naive implementation of the mask matrix $\mathbf{M}$ would entail storing an additional 1-bit mask for

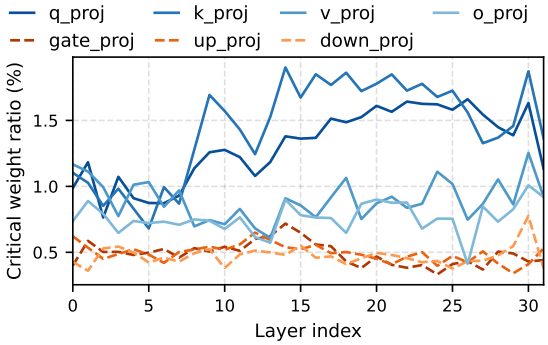


Figure 5: Critical weight ratios selected by the Kneedle algorithm per sublayer across 32 transformer layers of LLaMA-3-8B ($\rho = 0.1$). Solid lines: attention projections; dashed lines: MLP projections. Each sublayer exhibits a distinct per-layer ratio profile, in contrast to fixed-ratio methods.

every 8-dimensional vector, incurring significant bit overhead, especially given that E_8 quantization already achieves high compression, i.e., 16 bits per 8 weights. To avoid this overhead, we integrate the binary mask into the existing codebook index space. We utilize the fact that our codebook C contains 2^{16} codes, yet the probability of using the final code is statistically negligible in practical scenarios. Thus, we reserve the last index (i.e., $2^{16} - 1$) of C to serve as a special marker for unquantized vectors. When a vector $\mathbf{v}_{b,j}$ is marked for preservation, i.e., $\mathbf{M}_{8b:8(b+1),j} = \mathbf{1}$, its corresponding codebook index in the compressed structure is set to this dedicated index. This approach allows us to encode the preservation information without any bit overhead, as the single reserved index effectively replaces the need for a separate mask matrix storage.

4.4 The EPTQ Algorithm: Putting It Together

The EPTQ algorithm integrates all the components introduced in the previous sections. The entire procedure is detailed in Algorithm 3.

EPTQ takes the weight matrix $\mathbf{W}$, the layer input $\mathbf{X}$ obtained from the calibration dataset, and the codebook C generated as in Section 4.1.1 to yield the quantized weight matrix $\widehat{\mathbf{W}}$. The weight matrix $\mathbf{W}$ is first decomposed into the normalized weights $\mathbf{W}'$ and the scale vectors $\mathbf{g}$ and $\mathbf{h}$ using Algorithm 1, which is the weight scale normalization procedure described in Section 4.2 (line 3). Instead of directly quantizing $\mathbf{W}$, EPTQ quantizes the normalized matrix $\mathbf{W}'$. A crucial detail in quantizing $\mathbf{W}'$ is the use of the effective input $\mathbf{X}' = \text{diag}(\mathbf{h}) \cdot \mathbf{X}$ for calculating the Hessian inverse, instead of $\mathbf{X}$ (lines 4-6). This is because the original layer computation $\mathbf{Y} = \mathbf{W}\mathbf{X}$ can be reformulated as follows:

$$\begin{aligned}\mathbf{Y} = \mathbf{W}\mathbf{X} &= (\mathbf{W}' \odot (\mathbf{g}\mathbf{h}^\top))\mathbf{X} \\ &= \text{diag}(\mathbf{g})\mathbf{W}'\text{diag}(\mathbf{h})\mathbf{X} \\ &= \text{diag}(\mathbf{g})(\mathbf{W}'\mathbf{X}')\end{aligned}$$

showing that the effective input of $\mathbf{W}'$ is $\mathbf{X}'$.

For the critical weight preservation, described in Section 4.3, EPTQ prepares the mask matrix $\mathbf{M}$, which identifies the parameters

Algorithm 3: The EPTQ algorithm

Input: The weight matrix $\mathbf{W}$, the calibration dataset $\mathbf{X}$, the codebook C, search ratio ρ, codebook radius r, block size B

Output: The quantized weight matrix $\widehat{\mathbf{W}}$

1 $\widehat{\mathbf{W}} \leftarrow \mathbf{0}_{d_{\text{row}} \times d_{\text{col}}}$ // Quantized weights
2 $\mathbf{E} \leftarrow \mathbf{0}_{d_{\text{row}} \times B}$
3 $\mathbf{W}', \mathbf{g}, \mathbf{h} \leftarrow \text{IterativeWeightScaling}(\mathbf{W})$ // Algorithm 1
4 $\mathbf{X}' \leftarrow \text{diag}(\mathbf{h}) \cdot \mathbf{X}$
5 $\mathbf{H}^{-1} \leftarrow (2\mathbf{X}'\mathbf{X}'^{\top} + \lambda \mathbf{I})^{-1}$ // Hessian inverse
6 $\mathbf{H}^{-1} \leftarrow \text{Cholesky}(\mathbf{H}^{-1})$ // Cholesky decomposition
7 $\mathbf{M} \leftarrow \text{CriticalVectorMasking}(\mathbf{W}', \mathbf{X}', C, \rho, r)$ // Algorithm 2
8 **for** $i = 0, B, 2B, \cdots, d_{col} - B$ **do**
9 **for** $j = i, \cdots, i + B - 1$ **do**
10 $\alpha \leftarrow q_{0.95}(|\mathbf{W}'_{:,j}|)/r$
11 **for** $b = 0, \cdots, d_{row}/8 - 1$ **do**
12 $\mathbf{v} \leftarrow \mathbf{W}'_{8b:8(b+1),j}$
13 $\widehat{\mathbf{W}}_{8b:8(b+1),j} \leftarrow \text{fe8quant}(\mathbf{v}, \alpha, C)$ // Eq. (3)
14 $\widehat{\mathbf{W}}_{:,j} \leftarrow \mathbf{M}_{:,j} \odot \mathbf{W}'_{:,j} + (1 - \mathbf{M}_{:,j}) \odot \widehat{\mathbf{W}}_{:,j}$
15 $\mathbf{E}_{:,j-i} \leftarrow (\mathbf{W}'_{:,j} - \widehat{\mathbf{W}}_{:,j})/\mathbf{H}^{-1}_{jj}$
16 $\mathbf{W}'_{:,j:(i+B)} \leftarrow \mathbf{W}'_{:,j:(i+B)} - \mathbf{E}_{:,j-i}\mathbf{H}^{-1}_{j,j:(i+B)}$
17 $\mathbf{W}'_{:,(i+B):} \leftarrow \mathbf{W}'_{:,(i+B):} - \mathbf{E}\mathbf{H}^{-1}_{i:(i+B),(i+B):}$
18 $\widehat{\mathbf{W}} \leftarrow \widehat{\mathbf{W}} \odot (\mathbf{g}\mathbf{h}^{\top})$

to preserve, using Algorithm 2 (line 7). Then, the mask is applied during the column-wise quantization process of $\mathbf{W}'$. For each column j, the initial quantization $\widehat{\mathbf{W}}_{:,j}$ of $\mathbf{W}'_{:,j}$ is computed by the fe8quant function in Eq. (3) (lines 10–13). The mask $\mathbf{M}$ is then used to selectively update $\widehat{\mathbf{W}}_{:,j}$ with $\mathbf{W}'_{:,j}$ (line 14). The subsequent steps compute the error and update the remaining unquantized weights (lines 15–17). Finally, the resulting quantized matrix $\widehat{\mathbf{W}}$ is scaled back to get the final output quantized matrix as $\widehat{\mathbf{W}} \leftarrow \widehat{\mathbf{W}} \odot (\mathbf{g}\mathbf{h}^{\top})$ (line 18).

5 Experiments

In this section, we evaluate EPTQ by answering the following questions:

Q1. How does EPTQ compare to baselines in model quality (perplexity and zero-shot accuracy) and practical efficiency (quantization time and inference throughput) (Section 5.2)?

Q2. How do Weight Scale Normalization and Adaptive Critical Weight Preservation each contribute to model quality, and how does adaptive CWP compare to fixed-ratio approaches (Sections 5.3.1 and 5.3.2)?

Q3. How sensitive is EPTQ to the search ratio ρ and the codebook radius r (Section 5.3.3)?

5.1 Experimental Settings

5.1.1 Evaluation metrics. We evaluate the quantized models using three metrics: *Perplexity (PPL)* for language modeling capability, *zero-shot classification accuracy* for downstream task generalization, and *practical efficiency* in terms of quantization time and inference throughput. PPL is reported as token-level perplexity. PPL and zero-shot accuracy are measured using the widely-adopted *lm-evaluation-harness* library (v0.4.4) [10].

5.1.2 Models and datasets. We evaluate EPTQ on models from the Llama-2 [22] and Llama-3 [11] families: Llama-2-7B, Llama-2-13B, Llama-2-70B, and Llama-3-8B. For perplexity evaluation, we measure token-level PPL on Wikitext-2 [16] and C4 [17]. For zero-shot classification, we evaluate all four models on a suite of common-sense reasoning benchmarks: PIQA [2], ARC-easy, ARC-challenge [4], HellaSwag [28], and WinoGrande [18]. Following standard evaluation protocols, we report length-normalized accuracy for PIQA, ARC, and HellaSwag, and standard accuracy for WinoGrande. We use 128 calibration samples from C4 [17] following GPTQ [9], with each sequence truncated to 2048 tokens.

5.1.3 Baselines. We compare EPTQ against two categories of 2-bit PTQ baselines. For scalar quantization, we include GPTQ [9] and AWQ [13] on Llama-2 models to illustrate the performance gap between scalar and vector quantization at the extreme 2-bit regime. For GPTQ inference throughput, we use the Triton kernel backend from GPTQModel. For vector quantization, we include VPTQ [15] and QTIP [24] across all models as the primary comparison targets, representing the current state of the art in 2-bit accuracy, having demonstrated superior performance over prior VQ methods such as QuIP# [23], AQLM [6], and GPTVQ [25]. For VPTQ, we use the quantization configurations recommended in the original paper for Llama-2 models. For Llama-3-8B, the paper-recommended configuration produces severely degraded results due to a known code issue; we therefore use an alternative configuration (v=8, k=65536) for this model. All methods are evaluated without fine-tuning. All experiments are conducted using a single NVIDIA H100 GPU.

5.2 Model Quality and Efficiency

Table 1 reports token-level PPL on Wikitext-2 and C4, zero-shot classification accuracy on five common-sense reasoning benchmarks, and practical efficiency metrics for 2-bit quantized models. We evaluate EPTQ in two variants: E8, which uses the standard E_8 lattice codebook, and FE8, which uses the proposed Factored-E_8 codebook (Section 4.1).

Accuracy. Scalar quantization methods (GPTQ, AWQ) collapse catastrophically at 2-bit, with AWQ yielding perplexity in the hundreds of thousands on Llama-2 models. All three VQ methods (VPTQ, QTIP, and EPTQ) maintain meaningful accuracy. EPTQ consistently surpasses VPTQ across all models on both PPL and zero-shot accuracy. Against QTIP, EPTQ achieves competitive results on Llama-2 models: EPTQ (E8, ρ=0.1) attains a lower Wikitext-2 PPL than QTIP on Llama-2-7B (7.28 vs. 7.34) and the highest zero-shot average on Llama-2-13B (67.53 vs. 67.12), while matching QTIP on Llama-2-13B PPL (6.11 vs. 6.10). On Llama-2-70B and Llama-3-8B, QTIP achieves higher accuracy at comparable bit-widths.

Quantization efficiency. EPTQ (FE8) substantially reduces quantization time relative to both QTIP and VPTQ across all models. On Llama-2-13B, EPTQ FE8 completes quantization in 2,115 s, which is 6.4× faster than QTIP (13,531 s) and 2.3× faster than VPTQ (4,842 s). The advantage is even more pronounced on larger models: on Llama-2-70B, EPTQ FE8 requires 8,376 s versus 57,205 s for QTIP (6.8×) and 28,866 s for VPTQ (3.4×).

Table 1: 2-bit quantization results. Perplexity (PPL, ↓), zero-shot accuracy (%, ↑), quantization time (Qt., s, ↓), and decode throughput (Dt., tok/s, ↑). bpw: bits per weight (Appendix B). E8: standard E_8 lattice codebook; FE8: Factored-E_8 (Section 4.1). GPTQ and AWQ are scalar baselines on Llama-2 only. ‡QTIP inference throughput unavailable for Llama-2-13B (no dedicated 2-bit kernel support). †FP16 throughput for Llama-2-70B is not measured as the model does not fit on a single GPU. §AWQ inference throughput is not measured as no dedicated 2-bit kernel is available.

			PPL↓		Zero-shot Accuracy (%)↑						Efficiency	
Model	Method	bpw	Wiki2	C4	PIQA	ARC-e	ARC-c	HellaS	WinoG	Avg.	Qt.(s)	Dt.(tok/s)
Llama-2-7B	Full	16	5.47	6.97	79.11	76.26	46.25	76.00	69.14	69.35	–	187.1
	GPTQ	2.25	83.12	54.30	54.52	30.05	25.09	33.32	51.78	38.95	512	58.65
	AWQ	2.25	222150	166995	50.71	25.93	26.71	26.09	50.04	35.90	**419**	§–
	VPTQ	2.01	9.12	10.13	73.67	60.31	34.81	62.63	64.72	59.23	2,626	18.06
	QTIP	2.01	7.34	8.79	76.71	**67.09**	**39.25**	68.23	66.30	**63.52**	6,267	180.5
	EPTQ (E8, ρ=.02)	2.02	7.43	8.94	76.06	66.16	37.63	68.18	65.27	62.66	2,986	100.9
	EPTQ (E8, ρ=.1)	2.09	**7.28**	**8.76**	76.12	65.36	39.16	**68.29**	64.80	62.75	2,599	99.3
	EPTQ (FE8, ρ=.02)	2.03	7.61	9.14	74.97	63.85	36.95	66.89	65.19	61.57	1,240	**235.2**
	EPTQ (FE8, ρ=.1)	2.09	7.45	8.93	**76.77**	65.24	37.80	67.35	**66.46**	62.72	1,393	226.5
Llama-2-13B	Full	16	4.88	6.46	80.52	79.42	49.06	79.38	72.14	72.10	–	104.4
	GPTQ	2.25	19.72	19.01	61.53	40.82	27.47	44.95	52.09	45.37	791	31.0
	AWQ	2.25	122749	95112	50.11	27.06	27.82	26.01	48.62	35.92	**758**	§–
	VPTQ	2.01	7.22	8.69	76.66	65.24	39.33	69.09	68.67	63.80	4,842	27.91
	QTIP	2.01	**6.10**	**7.61**	77.64	70.20	43.52	73.45	**70.80**	67.12	13,531	‡–
	EPTQ (E8, ρ=.02)	2.02	6.21	7.75	77.64	**72.90**	**44.37**	73.28	68.43	67.32	5,662	59.8
	EPTQ (E8, ρ=.1)	2.08	6.11	7.65	**78.45**	71.97	43.69	**74.10**	69.46	**67.53**	5,114	59.5
	EPTQ (FE8, ρ=.02)	2.02	6.43	7.92	77.64	70.33	43.43	72.35	69.38	66.62	2,115	142.3
	EPTQ (FE8, ρ=.1)	2.09	6.26	7.81	77.75	71.76	43.09	72.99	70.01	67.12	2,196	**144.4**
Llama-2-70B	Full	16	3.32	5.52	82.70	82.74	57.42	83.82	77.98	76.93	–	†–
	GPTQ	2.25	9.94	10.83	67.52	50.59	30.89	52.83	61.17	52.60	**3,298**	6.72
	AWQ	2.25	72456	64939	49.95	25.88	28.67	25.72	49.17	35.88	3,529	§–
	VPTQ	2.07	4.93	7.19	81.07	78.87	52.65	79.07	77.11	73.75	28,866	7.22
	QTIP	2.00	**4.26**	**6.14**	**82.43**	**80.13**	**54.95**	**80.85**	77.03	**75.08**	57,205	**38.9**
	EPTQ (E8, ρ=.02)	2.02	4.50	6.30	81.34	79.08	52.90	80.06	76.87	74.05	29,300	15.1
	EPTQ (E8, ρ=.1)	2.08	4.44	6.27	81.23	77.95	52.05	80.20	**77.66**	73.82	29,265	15.1
	EPTQ (FE8, ρ=.02)	2.02	4.59	6.39	81.56	79.17	53.33	79.66	77.27	74.19	8,376	36.4
	EPTQ (FE8, ρ=.1)	2.08	4.57	6.34	81.18	79.04	53.07	79.71	76.72	73.94	8,654	36.2
Llama-3-8B	Full	16	6.14	8.91	80.79	80.13	53.33	79.16	72.69	73.22	–	161.8
	VPTQ	2.27	13.01	17.10	72.03	61.41	36.18	65.13	66.77	60.30	17,352	32.03
	QTIP	2.01	**9.98**	**13.25**	**75.14**	**65.32**	**41.72**	**69.22**	**70.09**	**64.30**	8,074	164.7
	EPTQ (E8, ρ=.02)	2.03	11.27	14.32	73.34	58.00	35.75	65.73	63.61	59.29	3,328	82.2
	EPTQ (E8, ρ=.1)	2.10	10.49	13.66	74.48	63.55	38.31	67.34	66.54	62.04	3,250	80.9
	EPTQ (FE8, ρ=.02)	2.03	11.95	14.90	73.78	59.76	33.53	63.09	63.93	58.82	1,496	**193.0**
	EPTQ (FE8, ρ=.1)	2.10	11.10	14.15	73.88	61.91	36.09	65.80	66.61	60.86	**1,486**	188.8

Inference throughput. EPTQ FE8 achieves the highest decode throughput among quantized methods on all models except Llama-2-70B. On Llama-2-7B, EPTQ FE8 reaches 235.2 tok/s, which is 1.3× faster than QTIP (180.5), 13× faster than VPTQ (18.1), and even exceeds FP16 throughput (187.1 tok/s) thanks to cache-efficient codebook lookup. On Llama-3-8B, EPTQ FE8 achieves 193.0 tok/s versus QTIP's 164.7 and VPTQ's 32.0, again surpassing FP16 (161.8 tok/s). On Llama-2-70B, QTIP and EPTQ FE8 achieve comparable throughput (38.9 vs. 36.2 tok/s), both far exceeding VPTQ (7.2 tok/s).

E8 vs. FE8. The two variants offer a complementary tradeoff. FE8 reduces the codebook from 1 MB (E8) to 4 KB (256× smaller), enabling it to reside entirely in the GPU L1 cache. At the same ρ setting, FE8 reduces quantization time by 2.2–3.5× and more than doubles inference throughput (≈2.3×) relative to E8. The accuracy cost is modest: on Llama-2 models, Wikitext-2 PPL increases by 0.13–0.22 and zero-shot average changes by less than 0.5%p at ρ=0.1. Users can select the variant that best suits their accuracy or latency requirements.

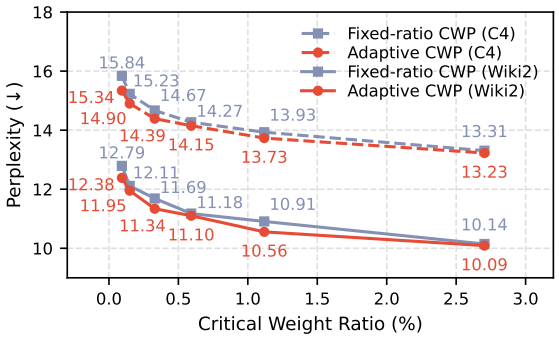


Figure 6: Adaptive CWP vs. fixed-ratio CWP on Llama-3-8B. At the same critical weight ratio, adaptive CWP achieves lower perplexity on both Wiki2 and C4, confirming that knee-point detection identifies critical weights more accurately than fixed-ratio CWP.

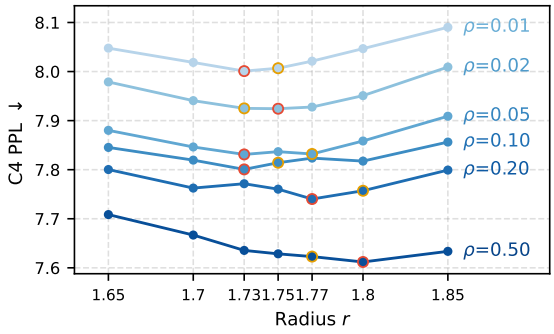


Figure 7: Hyperparameter sensitivity on Llama-2-13B. C4 PPL (↓) vs. codebook radius r for each search ratio ρ. Highlighted markers indicate the best (red outline) and second-best (orange outline) r per ρ.

5.3 Ablation Study

We conduct an ablation study covering three aspects: (1) the contribution of Weight Scale Normalization (WSN), (2) adaptive vs. fixed-ratio Critical Weight Preservation, and (3) sensitivity to the codebook radius r and search ratio ρ.

5.3.1 Weight Scale Normalization. Table 2 reports the contribution of WSN in EPTQ (FE8, $r = 1.75$) across all evaluated models and ρ settings. WSN consistently improves C4 PPL and zero-shot accuracy across all models. The gain is larger at ρ=.02 than at ρ=.1: at higher ρ, more critical weights are preserved in FP16, which partially compensates for row/column variance imbalance and reduces the marginal benefit of WSN. The effect is especially pronounced on Llama-3-8B (ΔC4 −3.4/−2.3, zero-shot +7.3/+4.2%), suggesting that its weight distributions exhibit stronger variance imbalance than Llama-2 models.

Table 2: Effect of Weight Scale Normalization (WSN). Each cell shows the gain from adding WSN, with absolute values (w/o→w/) in parentheses. FE8, $r = 1.75$.

Model	ρ	Δ C4 PPL (↓)	Δ Zero-shot (%↑)
Llama-2-7B	.02	−1.44 (10.58→9.14)	+3.0 (58.6→61.6)
	.1	−0.66 (9.59→8.93)	+1.7 (61.0→62.7)
Llama-2-13B	.02	−0.59 (8.51→7.92)	+1.5 (65.2→66.6)
	.1	−0.37 (8.18→7.81)	+0.9 (66.2→67.1)
Llama-3-8B	.02	−3.44 (18.34→14.90)	+7.3 (51.5→58.8)
	.1	−2.33 (16.48→14.15)	+4.2 (56.6→60.9)

5.3.2 Adaptive CWP vs. Fixed-ratio CWP. Figure 6 compares the proposed adaptive critical weight preservation (Section 4.3) against a fixed-ratio CWP baseline on Llama-3-8B, varying ρ from 0.01 to 0.5. To compare both methods at the same preservation budget, each point on the x-axis is constructed as follows: we first run adaptive CWP with a given search ratio ρ, which adaptively determines a different threshold per matrix; we then compute the resulting overall fraction of critical weights across all parameters, and use that global ratio as the fixed preservation ratio for the fixed-ratio CWP baseline. This ensures both methods preserve exactly the same total number of critical weights, isolating the effect of *how* those weights are selected. Adaptive CWP consistently achieves lower perplexity on both Wikitext-2 and C4 at every critical weight ratio tested, confirming that knee-point detection identifies critical weights more accurately than a uniform fixed ratio.

5.3.3 Sensitivity to r and ρ. Figure 7 shows C4 PPL across codebook radius r for each search ratio ρ on Llama-2-13B. PPL decreases monotonically as ρ increases as more critical weights preserved. Consistent with the theoretical prediction ($r \approx 1.77$), the optimal r lies near 1.77, yet it varies with ρ: higher ρ preserves more critical weights, which tend to reside far from the origin, so the remaining quantization targets cluster closer to the origin, favoring a larger r. Across all ρ values, however, PPL remains stable throughout $r \in [1.70, 1.80]$, indicating low sensitivity to the exact choice of r in this range. We therefore set $r = 1.75$ as the default: while not always the single best value, it consistently delivers near-optimal performance across all ρ settings.

6 Conclusion

We introduced EPTQ, a 2-bit post-training quantization framework offering two complementary variants. EPTQ (E8) achieves accuracy matching or surpassing state-of-the-art 2-bit VQ methods on Llama-2 models, while EPTQ (FE8) delivers the highest decode throughput among all evaluated methods on most models, exceeding FP16, with quantization completed up to 6.8× faster than competing approaches. These gains stem from three components: Factored-E_8 lattice quantization, lightweight scale normalization, and adaptive critical weight preservation with zero bit-overhead.

These results demonstrate that the accuracy-efficiency trade-off at the 2-bit limit is not fundamental: with careful codebook design, distribution alignment, and adaptive weight preservation, practical 2-bit compression can achieve both competitive model quality and superior deployment efficiency. Future directions include extending EPTQ to activation quantization and developing hardware-aware kernels to further amplify inference efficiency gains.

A Derivation of the Importance Score $S_{b,j}$

We detail the derivation of the importance score $S_{b,j}$ for the weight vector $\mathbf{v}_{b,j} = \mathbf{W}_{8b:8(b+1),j}$ using the method of Lagrange Multipliers. The score is defined as the minimum output loss change incurred when only this vector is quantized:

$$S_{b,j} = \min_{\widehat{\mathbf{W}}} \left\| \mathbf{W}\mathbf{X} - \widehat{\mathbf{W}}\mathbf{X} \right\|_F^2 \quad \text{s.t.} \quad \widehat{\mathbf{W}}_{8b:8(b+1),j} = \widehat{\mathbf{v}}^0_{b,j}$$

where $\widehat{\mathbf{v}}^0_{b,j} = \widehat{\mathbf{W}}^0_{8b:8(b+1),j}$ is the result of fe8quant$(\mathbf{v}_{b,j}, \alpha, C)$. Using the property of the Frobenius norm, $\|\mathbf{A}\mathbf{B}\|_F^2 = \mathrm{Tr}(\mathbf{A}\mathbf{B}\mathbf{B}^\top\mathbf{A}^\top)$, where $\mathbf{A} = \mathbf{W} - \widehat{\mathbf{W}}$ and $\mathbf{B} = \mathbf{X}$, we obtain:

$$S_{b,j} = \min_{\widehat{\mathbf{W}}} \mathrm{Tr}\left((\mathbf{W} - \widehat{\mathbf{W}})\mathbf{X}\mathbf{X}^\top (\mathbf{W} - \widehat{\mathbf{W}})^\top \right)$$

We define the error matrix $\mathbf{E} = \mathbf{W} - \widehat{\mathbf{W}}$, the Hessian $\mathbf{H} = 2\mathbf{X}\mathbf{X}^\top$, and the quantization error vector $\mathbf{e}^0_{b,j} = \mathbf{v}_{b,j} - \widehat{\mathbf{v}}^0_{b,j}$. The problem is rewritten as a standard constrained minimization of a quadratic form:

$$S_{b,j} = \min_{\mathbf{E}} \mathrm{Tr}\left(\frac{1}{2}\mathbf{E}\mathbf{H}\mathbf{E}^\top \right) \quad \text{s.t.} \quad \mathbf{E}_{8b:8(b+1),j} = \mathbf{e}^0_{b,j}$$

The trace of a matrix can be expressed as the sum of its row-vector quadratic forms. Since the error $\mathbf{E}$ is non-zero only in the rows $i = 8b, \cdots 8b+7$ due to the constraint, the summation simplifies:

$$S_{b,j} = \min_{\mathbf{E}} \sum_{i=8b,\cdots,8b+7} \frac{1}{2}\mathbf{E}_{i,:}\mathbf{H}\mathbf{E}_{i,:}^\top \quad \text{s.t.} \quad \mathbf{E}_{8b:8(b+1),j} = \mathbf{e}^0_{b,j}$$

Leveraging the row independence of the compensation mechanism, the minimization operator is exchanged with the summation, allowing us to solve 8 independent minimization problems, one for each row:

$$S_{b,j} = \sum_{i=8b,\cdots,8b+7} \min_{\mathbf{E}_{i,:}} \left(\frac{1}{2}\mathbf{E}_{i,:}\mathbf{H}\mathbf{E}_{i,:}^\top \right) \quad \text{s.t.} \quad \mathbf{E}_{i,j} = \mathbf{E}^0_{i,j} \tag{9}$$

where $\mathbf{E}^0_{i,j}$ is the $(i-8b)$-th component of the error vector $\mathbf{e}^0_{b,j}$. For each row i, we apply the method of Lagrange Multipliers to find the minimum of the quadratic form $L_i = \frac{1}{2}\mathbf{E}_{i,:}\mathbf{H}\mathbf{E}_{i,:}^\top$ subject to the linear constraint $\mathbf{E}_{i,:}\mathbf{u}_j - \mathbf{E}^0_{i,j} = 0$. We define the Lagrangian $\mathcal{L}_i$:

$$\mathcal{L}_i(\mathbf{E}_{i,:}, \lambda_i) = \frac{1}{2}\mathbf{E}_{i,:}\mathbf{H}\mathbf{E}_{i,:}^\top + \lambda_i(\mathbf{E}_{i,:}\mathbf{u}_j - \mathbf{E}^0_{i,j})$$

where $\mathbf{u}_j$ is a vector with 1 at the j-th position and 0 elsewhere. The optimal condition requires the gradient to be zero: $\nabla\mathcal{L}_i = 0$. To find the optimal solution, we take the partial derivatives of $\mathcal{L}_i$ with respect to the variables $\mathbf{E}_{i,:}$ and λ_i, and set them to zero.

$$\frac{\partial \mathcal{L}_i}{\partial \mathbf{E}_{i,:}} = \mathbf{E}_{i,:}\mathbf{H} + \lambda_i\mathbf{u}_j^\top = 0$$

The error vector $\mathbf{E}_{i,:}$ that satisfy this optimal condition is:

$$\mathbf{E}^*_{i,:} = -\lambda_i\mathbf{u}_j^\top\mathbf{H}^{-1} \tag{10}$$

We also take the partial derivative of the Lagrangian with respect to λ_i:

$$\frac{\partial \mathcal{L}_i}{\partial \lambda_i} = \mathbf{E}_{i,:}\mathbf{u}_j - \mathbf{E}^0_{i,j} = 0 \tag{11}$$

Substituting Eq. (10) into Eq. (11):

$$-\lambda_i\mathbf{u}_j^\top\mathbf{H}^{-1}\mathbf{u}_j - \mathbf{E}^0_{i,j} = 0$$

Since $\mathbf{u}_j^\top\mathbf{H}^{-1}\mathbf{u}_j$ is the scalar element $[\mathbf{H}^{-1}]_{j,j}$:

$$-\lambda_i[\mathbf{H}^{-1}]_{j,j} - \mathbf{E}^0_{i,j} = 0$$

The Lagrangian multiplier λ_i that satisfy this optimal condition is:

$$\lambda_i^* = -\frac{\mathbf{E}^0_{i,j}}{[\mathbf{H}^{-1}]_{j,j}}$$

The minimum loss $\min L_i$ is found by substituting $\mathbf{E}^*_{i,:}$ into the objective function L_i.

$$\begin{aligned}
\min L_i &= \frac{1}{2}\mathbf{E}^*_{i,:}\mathbf{H}(\mathbf{E}^*_{i,:})^\top \\
&= \frac{1}{2}(-\lambda_i^*\mathbf{u}_j^\top)(\mathbf{E}^*_{i,:})^\top \quad (\text{using } \mathbf{E}^*_{i,:}\mathbf{H} = -\lambda_i^*\mathbf{u}_j^\top) \\
&= \frac{1}{2}(-\lambda_i^*\mathbf{E}^0_{i,j}) \quad (\text{using } \mathbf{E}^*_{i,:}\mathbf{u}_j = \mathbf{E}^0_{i,j}) \\
&= \frac{1}{2}\left(-\left(-\frac{\mathbf{E}^0_{i,j}}{[\mathbf{H}^{-1}]_{j,j}} \right) \right)\mathbf{E}^0_{i,j} \\
&= \frac{[\mathbf{E}^0_{i,j}]^2}{2[\mathbf{H}^{-1}]_{j,j}}
\end{aligned}$$

Substituting $\mathbf{H} = 2\mathbf{X}\mathbf{X}^\top$ back into the result, which implies $[\mathbf{H}^{-1}]_{j,j} = \frac{1}{2}[\mathbf{X}\mathbf{X}^\top]^{-1}_{j,j}$:

$$\begin{aligned}
S_{b,j} &= \sum_{i=8b,\cdots,8b+7} \frac{[\mathbf{E}^0_{i,j}]^2}{2[\mathbf{H}^{-1}]_{j,j}} \\
&= \sum_{i=8b,\cdots,8b+7} \frac{[\mathbf{E}^0_{i,j}]^2}{2\left(\frac{1}{2}[\mathbf{X}\mathbf{X}^\top]^{-1}_{j,j}\right)} \\
&= \frac{\sum_{i=8b,\cdots,8b+7}[\mathbf{E}^0_{i,j}]^2}{[\mathbf{X}\mathbf{X}^\top]^{-1}_{j,j}}
\end{aligned}$$

Since the summation is the ℓ_2-norm squared of the error vector $\mathbf{e}^0_{b,j} = \mathbf{v}_{b,j} - \widehat{\mathbf{v}}_{b,j}$:

$$S_{b,j} = \frac{\left\|\mathbf{v}_{b,j} - \widehat{\mathbf{v}}_{b,j}\right\|_2^2}{[\mathbf{X}\mathbf{X}^\top]^{-1}_{j,j}} \tag{12}$$

This concludes the derivation.

B Bits-per-Weight Calculation

Each quantized 8-dimensional vector occupies 16 bits (2 bpw); each preserved vector occupies $16 + 128 = 144$ bits (18 bpw: reserved codebook index plus FP16 values). Scale vectors $\mathbf{g} \in \mathbb{R}^{d_{\text{row}}}$ and $\mathbf{h} \in \mathbb{R}^{d_{\text{col}}}$ (Section 4.2) add $16(d_{\text{row}} + d_{\text{col}})$ bits per matrix. For preservation fraction f (Algorithm 2):

$$\begin{aligned}
\text{bpw} &= (1-f) \times 2 + f \times 18 + 16\left(\frac{1}{d_{\text{row}}} + \frac{1}{d_{\text{col}}}\right) \\
&= 2 + 16f + 16\left(\frac{1}{d_{\text{row}}} + \frac{1}{d_{\text{col}}}\right)
\end{aligned}$$

The scale term is <0.01 for typical LLM matrices ($d \sim 4096$–11008). The bpw values in Table 1 vary across rows because the knee-point threshold selects a different f for each model and ρ setting.

References

[1] Saleh Ashkboos, Amirkeivan Mohtashami, Maximilian L Croci, Bo Li, Martin Jaggi, Dan Alistarh, Torsten Hoefler, and James Hensman. 2024. QuaRot: Outlier-Free 4-Bit Inference in Rotated LLMs. In *NeurIPS*.

[2] Yonatan Bisk, Rowan Zellers, Ronan Le Bras, Jianfeng Gao, and Yejin Choi. 2020. PIQA: Reasoning about Physical Commonsense in Natural Language. In *AAAI*. 7432–7439.

[3] Jerry Chee, Yaohui Cai, Volodymyr Kuleshov, and Christopher M De Sa. 2023. QuIP: 2-Bit Quantization of Large Language Models With Guarantees. In *NeurIPS*. 4396–4429.

[4] Peter Clark, Isaac Cowhey, Oren Etzioni, Tushar Khot, Ashish Sabharwal, Carissa Schoenick, and Oyvind Tafjord. 2018. Think you have Solved Question Answering? Try ARC, the AI2 Reasoning Challenge. *arXiv preprint arXiv:1803.05457* (2018).

[5] Tim Dettmers, Ruslan Svirschevski, Vage Egiazarian, Denis Kuznedelev, Elias Frantar, Saleh Ashkboos, Alexander Borzunov, Torsten Hoefler, and Dan Alistarh. 2023. SpQR: A Sparse-Quantized Representation for Near-Lossless LLM Weight Compression. In *NeurIPS*.

[6] Vage Egiazarian, Andrei Panferov, Denis Kuznedelev, Elias Frantar, Artem Babenko, and Dan Alistarh. 2024. Extreme Compression of Large Language Models via Additive Quantization. In *ICLR*.

[7] Elias Frantar and Dan Alistarh. 2022. Optimal Brain Compression: A Framework for Accurate Post-Training Quantization and Pruning. In *NeurIPS*. 4475–4488.

[8] Elias Frantar and Dan Alistarh. 2022. Optimal Brain Compression: A Framework for Accurate Post-Training Quantization and Pruning. In *NeurIPS*, S. Koyejo, S. Mohamed, A. Agarwal, D. Belgrave, K. Cho, and A. Oh (Eds.), Vol. 35. 4475–4488.

[9] Elias Frantar, Saleh Ashkboos, Torsten Hoefler, and Dan Alistarh. 2023. GPTQ: Accurate Post-Training Quantization for Generative Pre-trained Transformers. In *ICLR*.

[10] Leo Gao, Jonathan Tow, Baber Abbasi, Stella Biderman, Sid Black, Anthony DiPofi, Charles Foster, Laurence Golding, Jeffrey Hsu, Alain Le Noac'h, Haonan Li, Kyle McDonell, Niklas Muennighoff, Chris Ociepa, Jason Phang, Laria Reynolds, Hailey Schoelkopf, Aviya Skowron, Lintang Sutawika, Eric Tang, Anish Thite, Ben Wang, Kevin Wang, and Andy Zou. 2023. A Framework for Few-Shot Language Model Evaluation. *Zenodo* (2023). doi:10.5281/zenodo.10256836 Software version v0.4.0.

[11] Aaron Grattafiori, Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Lesechat, Aiko Asapur, Alex Schatz, Jane Dwivedi-Yu, et al. 2024. The Llama 3 Herd of Models. *arXiv preprint arXiv:2407.21783* (2024).

[12] B. Hassibi, D. G. Stork, and G. J. Wolff. 1993. Optimal Brain Surgeon and General Network Pruning. In *ICNN*. 293–299.

[13] Ji Lin, Jiaming Tang, Haotian Tang, Shang Yang, Wei-Ming Chen, Wei-Chen Wang, Guangxuan Xiao, Xingyu Dang, Chuang Gan, and Song Han. 2024. AWQ: Activation-Aware Weight Quantization for LLM Compression and Acceleration. In *MLSys*.

[14] Han Liu, Haotian Gao, Xiaotong Zhang, Changya Li, Feng Zhang, Wei Wang, Fenglong Ma, and Hong Yu. 2025. SEPTQ: A Simple and Effective Post-Training Quantization Paradigm for Large Language Models. In *KDD*. 812–823.

[15] Yifei Liu, Jicheng Wen, Yang Wang, Shengyu Ye, Li Lyna Zhang, Ting Cao, Cheng Li, and Mao Yang. 2024. VPTQ: Extreme Low-bit Vector Post-Training Quantization for Large Language Models. In *EMNLP*.

[16] Stephen Merity, Caiming Xiong, James Bradbury, and Richard Socher. 2017. Pointer Sentinel Mixture Models. In *ICLR*.

[17] Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J. Liu. 2020. Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer. *J. Mach. Learn. Res.* 21, 140 (2020), 1–67.

[18] Keisuke Sakaguchi, Ronan Le Bras, Chandra Bhagavatula, and Yejin Choi. 2021. WinoGrande: An Adversarial Winograd Schema Challenge at Scale. *Commun. ACM* 64, 9 (2021), 99–106.

[19] Ville Satopaa, Jeannie Albrecht, David Irwin, and Barath Raghavan. 2011. Finding a "Kneedle" in a Haystack: Detecting Knee Points in System Behavior. In *31st International Conference on Distributed Computing Systems Workshops*. IEEE, 166–171.

[20] Wenqi Shao, Mengzhao Chen, Zhaoyang Zhang, Peng Xu, Lirui Zhao, Zhiqian Li, Kaipeng Zhang, Peng Gao, Yu Qiao, and Ping Luo. 2024. OmniQuant: Omnidirectionally Calibrated Quantization for Large Language Models. In *ICLR*.

[21] Richard Sinkhorn and Paul Knopp. 1967. Concerning nonnegative matrices and doubly stochastic matrices. *Pacific J. Math.* 21, 2 (1967), 343–348.

[22] Hugo Touvron, Louis Lavril, Guillaume Izacard, Xavier Martinet, Marie-Anne Lachaux, Tim Muennich, Thibaut Saurette, Arturo Rivera, Priyanka Goyal, Spencer Hambro, et al. 2023. Llama 2: Open Foundation and Fine-Tuned Chat Models. *arXiv preprint arXiv:2307.09288* (2023).

[23] Albert Tseng, Jerry Chee, Qingyao Sun, Volodymyr Kuleshov, and Christopher De Sa. 2024. QuIP#: Even Better LLM Quantization with Hadamard Incoherence and Lattice Codebooks. In *ICML*.

[24] Albert Tseng, Qingyao Sun, David Hou, and Christopher De Sa. 2024. QTIP: Quantization with Trellises and Incoherence Processing. In *NeurIPS*. https://openreview.net/forum?id=7sdkLVuYCU

[25] Mart van Baalen, Andrey Kuzmin, Markus Nagel, Peter Couperus, Cédric Bastoul, Eric Flamand, Tijmen Blankevoort, and Max Welling. 2024. GPTVQ: The Blessing of Dimensionality for LLM Quantization. *arXiv preprint arXiv:2402.15319* (2024).

[26] Maryna S. Viazovska. 2017. The Sphere Packing Problem in Dimension 8. *Ann. Math.* (2017), 991–1015.

[27] Guangxuan Xiao, Ji Lin, Mickael Seznec, Hao Wu, Julien Demouth, and Song Han. 2023. SmoothQuant: Accurate and Efficient Post-Training Quantization for Large Language Models. In *ICML*.

[28] Rowan Zellers, Ari Holtzman, Yonatan Bisk, Ali Farhadi, and Yejin Choi. 2019. HellaSwag: Can a Machine Really Finish Your Sentence?. In *ACL*. 4791–4800.